

```

/*****
 * File: MajorityElement.hh
 * Author: Keith Schwarz (htiek@cs.stanford.edu)
 *
 * A one-
pass, linear time algorithm that returns the majority element of a
 * range of data. The majority element is an element that appears strict
 * more than half the time. For example, in the sequence
 *
 *           0 1 0 0 2 0 3
 *
 * The number 0 is a majority element, since it appears 4/7 times. Howev
 * in the sequence
 *
 *           0 1 0 0 2 0 3 3
 *
 * There is no majority element, since even though 0 occurs 4/8 times, th
 * isn't strictly greater than half the elements.
 *
 * The algorithm for finding the majority element is remarkably simple, b
 * correctness is not immediately obvious. The algorithm works as follow
 * each step, we maintain our "guess" of what the majority element will b
 * also a counter. We then scan across the array. At each point, if the
 * element matches our current guess we increment the counter, and otherw
 * we decrement it. If the counter is ever zero, then on the next elemen
 * change the counter to 1 and pick the next element as our guess. Final
 * output the guessed element. For example, here is the algorithm runnin
 * the earlier input. The topmost row shows the input, below that our gu
 * and below that the counter:
 *
 * INPUT      0 1 0 0 2 0 3
 * GUESS      ? 0 ? 0 0 0 0
 * COUNTER    0 1 0 1 2 1 2 1
 *
 * Since our guess at the end is zero, we output zero. This algorithm is
 * due to Boyer and Moore and is described in their paper "MJRTY - A Fast
 * Majority Vote Algorithm."
 *
 * There are many ways to think about why this algorithm works. One good
 * intuition is to think of the algorithm as breaking the input down into
 * of stretches of consecutive copies of particular values. Incrementing
 * counter then corresponds to marking that multiple copies of the same v
 * were found, while decrementing it corresponds to some other sequences
 * values "canceling out" the accumulation of values of a particular type
 *
 * A formal proof of correctness of this algorithm (based on the proof in
 * Boyer and Moore's paper) relies on a key lemma. In this section, we'l
 * let C be the number that is currently a candidate for the majority ele
 * K be its count after some number of steps, and N be the number of tota
 * elements.
 *
 * Lemma 1: For any i,  $1 \leq i \leq N$ , after i steps of the algorithm, the
 * elements in the range [1, i] can be rearranged into two groups A and B su
 * that A is K copies of C, and B is a collection of elements with at mos
 *  $i / 2$  copies of any one element.
 *
 * Let's hold off of the proof of this lemma for now, and show that if it
 * holds and there is a majority element, the algorithm must be correct.
 * the above lemma, note that when the algorithm terminates, there must b
 * element C that was chosen with some count K. Assume for the sake of
 * contradiction that C is not the majority element; then there is some o

```

